PROJECT 5 • CAPSTONE LAB

Full AWS Infrastructure with Terraform

A secure, scalable, production-style stack: networking, compute, load balancing, database, and DNS — all as code.

VPC · Bastion · ASG · ALB · RDS · S3 · Route 53

SCENARIO

What the Company Needs

A secure, scalable AWS infrastructure built entirely with Terraform.

VPC with Public & Private Subnets

Segmented network foundation for the whole stack.

Bastion Host for SSH Access

A single, controlled entry point into private resources.

Auto Scaling for App Servers

Capacity that grows and shrinks with demand.

Application Load Balancer

Distributes traffic for high availability.

Amazon RDS (MySQL)

Managed, durable database storage.

Route 53 Custom Domain

Friendly DNS name resolving to the app.

ARCHITECTURE

How the Pieces Fit Together

Internet — Route 53 — ALB

S3 Bucket

Static assets

VPC — 10.0.0.0/16

Public Subnet

- Bastion Host
- ALB target

Private Subnet

- Auto Scaling Group · min 2 / desired 3 / max 5

Why the Bastion Matters

Admins SSH into the bastion (public subnet), then hop to private instances — no app server is ever exposed directly to the internet.

Amazon RDS (MySQL)

Private subnet only — no direct internet access

Reached only from the ASG instances

STEP 1

Create VPC with Public & Private Subnets

Using the official terraform-aws-modules/vpc/aws module

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"

  cidr = "10.0.0.0/16"

  enable_dns_hostnames = true
  enable_dns_support   = true
}
```

Community module

Reuses a battle-tested module instead of hand-rolling subnets and route tables.

DNS enabled

enable_dns_hostnames / enable_dns_support let instances resolve friendly hostnames.

Result

A VPC with networking ready for every resource that follows.

STEP 2

Deploy a Bastion Host for SSH Access

A single, controlled entry point into the private network

```
resource "aws_instance" "bastion" {
  ami           = "ami-123456"
  instance_type = "t2.micro"

  subnet_id = module.vpc.public_subnets[0]
  key_name  = var.ssh_key_name
}
```

`public_subnets[0]`

Lives in the public subnet so it's reachable from the internet.

`key_name`

SSH key pair — no passwords, key-based access only.

`Purpose`

Every hop into the private network goes through this one hardened host.

STEP 3

Create an Auto Scaling Group

Application servers that scale with demand, in the private subnets

```
resource "aws_autoscaling_group" "web_asg" {
  desired_capacity = 3
  max_size          = 5
  min_size          = 2

  vpc_zone_identifier = module.vpc.private_subnets
}
```

`min 2 / desired 3 / max 5`

Always at least 2 instances; scales up to 5 under load.

`private_subnets`

App servers never get a public IP — traffic only arrives via the ALB.

`Elastic capacity`

Instances are added or removed automatically as demand changes.

STEP 4

Deploy an Application Load Balancer

Distributes incoming traffic across the Auto Scaling Group

```
resource "aws_lb" "app_lb" {
  name = "app-lb"

  internal           = false
  load_balancer_type = "application"
}
```

`internal = false`

The ALB is internet-facing — it's the front door for users.

`Targets the ASG`

Routes each request to a healthy instance in the private subnets.

`High availability`

If one instance fails, traffic shifts to the healthy ones automatically.

STEP 5

Configure Route 53 for a Custom Domain

Maps a friendly domain name to the load balancer

```
resource "aws_route53_record" "app_dns" {
  zone_id = var.zone_id
  name    = "app.example.com"
  type    = "A"

  alias {
    name                   = aws_lb.app_lb.dns_name
    evaluate_target_health = true
  }
}
```

Alias record

Points directly at the ALB's DNS name — no static IP to manage.

evaluate_target_health

Route 53 stops routing to the ALB if it becomes unhealthy.

Result

Users visit app.example.com instead of a raw AWS hostname.

VERIFICATION

Confirm Everything Is Running

Three checks to validate the stack end-to-end.

1

Check the Load Balancer

Confirms the ALB exists and is active.

```
$ aws elbv2 describe-load-balancers
```

2

Verify Auto Scaling

Confirms the ASG is running within its min/max bounds.

```
$ aws autoscaling describe-auto-scaling-groups
```

3

Test Bastion SSH Access

Confirms the entry point into the private network works.

```
$ ssh -i my-key.pem ec2-user@<bastion-host-ip>
```

PROJECT 5 SUMMARY

The Full Stack, as Code

Layered Network Security

Public subnet for the bastion & ALB; private subnets for app servers and RDS.

Elastic & Highly Available

The ASG scales compute, the ALB spreads traffic and absorbs failures.

One Domain, Fully Wired

Route 53 ties a custom domain straight to the load balancer.

End of Project 5